Assignment 2: Higher-Order Function and Untyped Lambda Calculus

CSC324H1, Fall 2025

Due Date: October 10, 2025 (11:59 pm)

Please read the instructions carefully before you start.

Overview

In this assignment, you will:

- Explore functional programming using higher-order functions;
- Extend the small-step evaluator with untyped lambda calculus constructs using nameless representation of variables;
- Implement an environment-based and big-step evaluator for the untyped lambda calculus.

Instructions

A starter project with code skeleton is provided to you. For Part 1, complete the functions marked with `error "TODO"` in the starter code. For Part 2, extend and modify the provided code as appropriate, and complete the functions with TODOs.

Run the tests provided to check your work. Note that your code will be graded based on additional hidden tests.

Please note that for all assignments:

- It is recommended to only submit all required files as specified on MarkUs.
- You are responsible for making sure that your code has no syntax errors or compile *errors*. If your file cannot be imported by another file, you may receive a grade of zero. Compiler *warnings* can help you locate potential issues, but they do not affect your grade.
- If you're not intending to complete one of the functions, do *not* remove the function signature. If you are including a partial solution, make sure it doesn't cause a compile error. If your partial solution causes compile errors, it's better to comment out your solution (but not the signature).
- Do not modify the `module ... where` and `import ...` lines of your code. These lines are crucial for your code to pass the tests.

Part 1: Higher-Order Functions (35 pts)

In this part, you will be working in the file `HigherOrderFunctions.hs`.

(a) Higher-Order Function: "Buy the Dip" (15 pts)

Mark has implemented an investment strategy: he looks at the price of the stock once a day, and starting from the second day, if the price is lower than what it was yesterday, he buys one stock at the price of the day.

Given a list of the stock price, you are to implement the function `totalCost :: [Int] -> Int` that calculates how much money Mark spent on buying stock for that period of time. Mark never buys on the first day. The price of the stock is always positive.

For this question, you are *not allowed* to use recursion or pattern matching explicitly. Use higher-order list functions instead.

(b) `mapTree` (5 pts)

Implement `mapTree`. The function takes a function `f` of type `a -> b`, and a `(BinaryTree a)` with data of type `a`. As output, `mapTree` applys `f` to every leaf and internal node of the binary tree.

(c) `transformTree` and Tree Traversal (10 pts)

Following up, there are more ways to transform a tree than simply mapping its internal data. We can go by further changing the output type all together.

Write the function `transformTree` that takes the following inputs:

- `f :: a -> b`: Transforms `(Leaf a)` to a value of type `b`;
- `g :: b -> a -> b -> b`: Transforms `(Branch l x r)` to a value of type `b` by taking the transformed left subtree as the first argument, the data `x` of type `a` in the node as the second argument, and the transformed right subtree as the third argument.
- `tree :: (BinaryTree a)`: The tree to be transformed.

The result of the transformation is a value of type `b`. Then, implement `preOrder`, `inOrder`, and `postOrder` by calling `transformTree`, so the functions transform a tree to a list of its data in the order of pre-order, in-order, and post-order tree traversal, respectively.

(d) Function as Output (5 pts)

A function with multiple arguments can be desugared into ones with a single argument. After the first argument is supplied, another function that takes the second argument is returned:

$$\lambda x\,.\,\underbrace{(\lambda y\,.\,e)}_{\text{Takes the second argument}}$$

Write two functions that transform a binary function between "taking arguments one at a time", and "taking both arguments together as a tuple":

- Implement `curry :: ((a, b) -> c) -> (a -> b -> c)`
- Implement `uncurry :: (a -> b -> c) -> ((a, b) -> c)`

Part 2: Evaluators for the Untyped Lambda Calculus (65 pts)

In this part of the assignment, you will be extending the evaluators from Assignment 1 to handle the additional language constructs (variables, lambda abstraction, applications), and evaluation under environment.

(a) Explanation of Starter Code

In class, you have seen two representations of lambda calculus:

- *Named representation* which uses explicit variable names
- *Nameless representation* which uses indices to refer to variables

Nameless Representation We extend the datatype `Term` from Assignment 1 to support nameless representation of lambda calculus:

```
data Term = ... | TmVar Int | TmAbs Term | TmApp Term Term
```

Named Representation We also introduce a new datatype `NamedTerm`, which represents the syntax of lambda calculus with explicit names. Most of the constructs are similar to their nameless representations. The new extensions are as follows:

```
type Identifier = String
data NamedTerm = ...
               | NTVar Identifier
               | NTAbs Identifier NamedTerm
               | NTApp NamedTerm NamedTerm
```

Values for Big-Step Semantics We extend the definitions of values with *closures*:

```
data Value = ... | Closure Env Identifier NamedTerm
```

A closure value of `(Closure env iden body)` represents a function that carries an environment `env`, with the name of parameter as `iden`, and the function body `body`.

Auxiliary Definitions We also introduce a few new types and datatypes across the source files:

```
type NamingContext = [Identifier]
type Env = [(Identifier, Value)]
```

Type	Note
`NamingContext`	The naming context as defined by Definition 6.1.3 in TAPL. It is a list of identifiers as string, with the k^{th} element of the list being the identifier introduced by the k^{th} enclosing λ.
`Env`	A list of identifier-value bindings.

(b) Working with Named and Nameless Representation

Translation between representations (`removeNames`) We start with a translation function from named to nameless representation. The function is declared in `DeBruijn.hs` with the following signature:

```
removeNames :: NamingContext -> NamedTerm -> Term
```

The function takes in a `NamingContext` and a `NamedTerm`. As output, it outputs a `Term` that is the nameless representation of the input `NamedTerm`. We do *not* test the situation where a variable cannot be found in the naming context.

Shifting (`shifting`) The shifting function is a commonly used utility function when dealing with nameless representation. The function is declared in `DeBruijn.hs` with the following signature:

```
shifting :: Int -> Int -> Term -> Term
```

The arguments in order are specified as follows:

	Type	Note
`d`	`Int`	The increment to be shifted by.
`c`	`Int`	The cutoff.
`t`	`Term`	The nameless lambda calculus term to be shifted.

The function outputs the shifted nameless term $\uparrow_c^d$ (`t`) as seen in class and TAPL.

Substitution (`subst`) The substitution function is the last piece of scaffolding before we can properly implement nameless representation. It is declared in `DeBruijn.hs` with the following signature:

```
subst :: Int -> Term -> Term -> Term
```

The arguments in order are specified as follows:

	Type	Note
`j`	`Int`	The index of the variables to be substituted.
`s`	`Term`	The term all matching variables are substituted to.
`t`	`Term`	The nameless lambda calculus term to be substituted.

The function outputs the substitution [`j` $\mapsto$ `s`]`t` as seen in class and TAPL.

(c) Small-Step Semantics with Nameless Representation

We are now ready to implement the small-step semantics of lambda calculus using the nameless representation.

Extend `oneStep` in `SmallStep.sh` that you have worked on in Assignment 1. The signature of the function did not change:

```
oneStep :: Term -> StepResult
```

However, you will have to handle more language constructs and evaluation rules.

Make sure `multiStep` is correct with respect to the new specifications.

(d) Big-Step Semantics with Named Representation

The big-step evaluator function is in `BigStep.hs` with the following signature:

```
bigStep :: Env -> NamedTerm -> EvalResult
```

The function takes an environment `env` and a `NamedTerm t` to be evaluated. It returns a result of type `EvalResult`, which is either an `(EvalOk v)` if `env` $\vdash$ `t` $\Downarrow$ `v` holds, or `(EvalError msg)` for some error message `msg`.

Your Tasks and Tips

1. (10 pts) Implement the translation function `removeNames` in `DeBruijn.hs`.
   - Your function should implement the translation as described in Section 6.1 in TAPL.
   - You are not required to consider the case where a variable refers to an identifier not in the naming context.
2. (10 pts) Implement the shifting function `shifting` in `DeBruijn.hs`.
   - Your function should essentially implement Definition 6.2.1 in TAPL.
   - You might find Chapter 7 of TAPL useful.
3. (10 pts) Implement the substitution function `subst` in `DeBruijn.hs`.
   - Your function should essentially implement Definition 6.2.4 in TAPL.
   - You might find Chapter 7 of TAPL useful.
4. (10 pts) Extend the one-step evaluator `oneStep` in `SmallStep.hs`.
   - The extensions should essentially implement `E-AppAbs` as described in Section 6.3 of TAPL and the other rules specified in Figure 5-3 of TAPL.
   - You might find Chapter 7 of TAPL useful.
   - *Do not* modify the given `multiStep`. However, make sure `multiStep` is correct with respect to the new specifications and your new implementation of `oneStep`.
   - Error messages are not tested.
5. (25 pts) Implement the big-step evaluator `bigStep` in `BigStep.hs`.
   - The big-step evaluator should essentially implement the big-step evaluation relation $E \vdash t \Downarrow v$ as presented in the Week 5 lecture, and the big-step rules for untyped arithmetic expressions on slide 39 of the Week 2 lecture, after extending them with the environment.
   - Error messages are not tested.